SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA17**

COURSE OUTCOME COVERED

CO2: *Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)*

Assessment Tool Weightage: 35%

Duration: 1 Hour

QUESTIONS: 5 **Total Marks:50**

Annexure A

Scenario Based Assignment

Code of Conduct:

I **Eshwanth C / 192321062** *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

Scenario Based Assignment

1. Scenario : A government deploys AI-powered drones to deliver emergency medicines in a flood-affected region. The environment is highly dynamic—roads may suddenly become inaccessible, and weather conditions affect travel cost. The drone has access to: A heuristic estimate (straight-line distance), Partial real-time updates about blocked paths and Variable movement costs depending on terrain and weather. However, due to urgency, the drone must quickly decide routes with minimum delay and risk.

Tasks:

i. Explain how Greedy Best-First Search would behave in this scenario and discuss its strengths and weaknesses under uncertainty

ii. Explain how A* would handle the same situation, including how it balances cost and heuristic

iii. Analyze the impact of heuristic accuracy on both algorithms

iv. Discuss how dynamic changes (blocked paths) affect search performance

v. Recommend the most suitable algorithm and justify your decision considering real-time constraints, optimality, and safety

2. **Scenario**: A smart city implements an AI system to optimize traffic signal timings across hundreds of intersections. The objective is to minimize Total waiting time, Fuel consumption and Traffic congestion. The system must continuously adjust signals based on traffic flow. However the search space is extremely large, traffic patterns change dynamically and the system may get stuck in locally optimal solutions

Tasks. Formulate this as an optimization problem and explain why traditional search is inefficient

i. Explain how Hill Climbing would work in this scenario and identify its limitations

ii. Discuss issues like local maxima, plateaus, and ridges with real-world interpretation

iii. Explain how Simulated Annealing or Genetic Algorithms can overcome these limitations

iv. Recommend the best approach and justify your choice based on scalability and adaptability

3. **Scenario:** An autonomous rover is deployed on Mars to explore unknown terrain. It must: navigate safely to collect samples, avoid hazards (craters, rocks), operate with limited sensing capability and make decisions without a complete map. The rover updates its knowledge as it explores, making this a real-time decision-making problem.

Tasks:

i. Explain why this problem requires an online search agent instead of offline search

ii. Analyze the challenges of partial observability and dynamic environments

iii. Describe how the rover builds and updates its internal model

iv. Compare online search with uninformed and informed search strategies

v. Justify the most suitable approach considering uncertainty, adaptability, and safety

4. **Scenario:** A university is designing an AI system to automatically generate exam timetables. The system must satisfy multiple constraints: No student should have overlapping exams, Limited number of exam halls, Certain subjects must be scheduled before others, Faculty availability constraints, Special accommodations for some students, The complexity increases as the number of courses and students grows.

Tasks :

Formulate the problem as a Constraint Satisfaction Problem (CSP)

i. Clearly define variables, domains, and constraints with explanation

ii. Explain how backtracking search works in this scenario

iii. Discuss how constraint propagation (e.g., forward checking) improves efficiency

iv. Analyze real-world challenges and justify the best strategy for scalability

5. **Scenario:** Strategic Game AI for Real-Time Decision Making (Minimax & Alpha-Beta Pruning) A gaming company is developing an AI opponent for a real-time strategy game. The AI must complete against human players, Make decisions within strict time limits, Evaluate complex game states with multiple possible moves, The decision tree is extremely large, making computation expensive.

Tasks:

i. Explain how the Minimax algorithm models this problem

ii. Analyze the computational challenges of large game trees

iii. Explain how Alpha-Beta pruning improves efficiency with detailed reasoning

iv. Design an evaluation function and justify the factors considered

v. Recommend a strategy for balancing optimality and real-time performance

Annexure A – Scenario Based Assignment (Answers)

Name: Eshwanth C **Reg No:** 192321062 **Course:** Artificial Intelligence (CSA17)

1. Emergency medicine drones

i. Greedy Best-First Search (GBFS)

- GBFS expands the node with the lowest h(n) (straight-line distance to the goal) and ignores the cost already spent.
- *Strengths:* very fast, low memory, heads straight to the destination, so it reacts quickly when time is critical.
- *Weaknesses:* ignores terrain and weather cost, so it can choose a short but dangerous route. It is not optimal and can be incomplete (loops into dead ends if a flooded road blocks the straight line). Under uncertainty it can commit to a path that is suddenly blocked.

ii. A*

- A* uses f(n) = g(n) + h(n). g(n) is the real accumulated cost (terrain difficulty, weather penalty); h(n) is the straight-line estimate to the goal.
- g stops the drone from taking cheap-looking but costly routes, h keeps the search focused on the goal. Straight-line distance never overestimates, so h is admissible and A* is complete and optimal on the known map.

iii. Impact of heuristic accuracy

- GBFS depends completely on h: a poor estimate (a straight line across a flooded zone) leads it straight into traps.
- A* tolerates a weak h because g corrects the choice; a more accurate (still admissible) h makes A* expand fewer nodes. If h overestimates, A* loses optimality.

iv. Effect of dynamic changes (blocked paths)

- A blocked path invalidates the current plan. GBFS can re-plan very cheaply from the new position but the new path may still be poor.
- A* costs more per re-plan but gives better routes; incremental variants (D* Lite, LPA*) reuse earlier work and update quickly. Frequent changes mean frequent re-planning, so search time matters.

v. Recommendation

- Use A* with re-planning (D* Lite / Replanning A*): it balances cost and heuristic, avoids high-risk terrain through g(n) and gives optimal or near-optimal routes with acceptable speed.
- GBFS is only a fast fallback if the time limit is extremely tight. For medicine delivery, safety and reliability matter more than raw speed.

2. Smart city traffic signals

Formulation as an optimization problem

- *State:* timing plan (green time, offset, cycle length) of every intersection. *Action:* change the timing of one or more signals.
- *Objective:* minimize Cost = w1·(total waiting time) + w2·(fuel consumption) + w3·(congestion).
- *Why traditional search is inefficient:* the space is exponential (e.g. 60 timing values at 500 intersections = 60^{500} states), traffic keeps changing so a precomputed path is useless, and there is no single goal state, only a best configuration. BFS, DFS and A* cannot handle this size.

i. Hill Climbing and its limitations

- Start from a timing plan, evaluate neighbours (change one signal) and move to the best one; stop when no neighbour is better.
- Limitations: stops at a local optimum, no memory, result depends on the start point, slow on flat regions.

ii. Local maxima, plateaus, ridges

- *Local maximum:* one corridor is perfectly synchronised but it worsens the neighbouring area; no single-signal change improves the total.
- *Plateau:* many timings give the same waiting time (e.g. light night traffic), so there is no gradient to follow.
- *Ridge:* improvement needs several signals to change together (a green wave along a road); changing one signal at a time makes things worse.

iii. Simulated Annealing / Genetic Algorithm

- *Simulated Annealing:* accepts a worse move with probability $e^{-\Delta E/T}$; T is high at first (exploration) and is lowered gradually (exploitation), which lets it escape local maxima.
- *Genetic Algorithm:* keeps a population of timing plans and improves it by selection, crossover and mutation. Parallel search covers more of the space and avoids local traps.

iv. Recommendation

- Genetic Algorithm (optionally combined with local search): scales to hundreds of intersections through parallel evaluation, and the population can adapt as traffic changes by restarting from good existing plans.
- Simulated Annealing is simpler but works on a single solution, so it adapts less well to continuously changing traffic.

3. Autonomous Mars rover

i. Why online search

- The map is unknown, so a complete plan cannot be computed in advance. An online agent interleaves planning and acting: move, sense, decide again. Offline search needs the full state space and all action outcomes beforehand.

ii. Partial observability and dynamic environment

- The rover sees only nearby terrain, so hazards beyond sensor range are unknown; sensor noise causes misjudged craters and rocks; dust storms and shifting sand change the terrain.
- Some actions are irreversible (falling into a crater), plans can become invalid, and the rover may loop without a memory of visited places.

iii. Building and updating the internal model

- The rover keeps a map (occupancy grid with unknown / free / hazard cells), visited states and costs. After every move it senses, updates the map, records result(s, a) = s' and updates cost estimates (as in LRTA*, which learns h(s) while exploring), then chooses the next move from the updated model.

iv. Comparison

- *Uninformed (BFS/DFS):* needs the whole map, ignores goal knowledge and explores blindly; physical backtracking wastes energy.
- *Informed (A*/Greedy):* goal-directed and faster but assumes known, stable terrain.
- *Online search (LRTA*, online DFS):* works with local knowledge, learns as it moves and handles uncertainty.

v. Justification

- Online informed search such as LRTA* or D* Lite: the heuristic keeps it moving to the goal, learned values adapt to newly seen terrain, a safety cost on hazards keeps it away from craters, and re-planning handles surprises.

4. University exam timetabling (CSP)

i. Formulation: variables, domains, constraints

- *Variables:* each exam E_i (one per course); optionally a hall variable H_i.
- *Domains:* all (day, time slot, hall) combinations, e.g. {Day1-9AM, Day1-2PM, …} × {Hall1 … HallN}.
- *Constraints:* (1) No overlap: E_i and E_j must have different slots if any student takes both. (2) Hall capacity: one exam per hall per slot and the students must fit the hall. (3) Precedence: slot(E_i) < slot(E_j) if E_i must come first. (4) Faculty availability: the slot must be in the invigilator's available set. (5) Accommodations: special-needs students get a particular hall or extra time.

ii. Backtracking search

- Assign an exam to a value, check the constraints and go deeper if consistent; on a conflict undo the last assignment and try the next value. It is DFS with constraint checking and finds a valid timetable or proves none exists.
- Heuristics help: MRV (exam with fewest legal slots first), degree heuristic, LCV for value ordering.

iii. Constraint propagation

- *Forward checking:* after assigning E_i, remove conflicting values from the domains of its neighbours; if a domain becomes empty, backtrack immediately.
- This detects failure early and prunes the tree; arc consistency (AC-3) prunes further, giving far fewer backtracks.

iv. Real-world challenges and best strategy

- *Challenges:* thousands of exams and students, conflicting constraints (a full solution may be impossible), last-minute changes.
- *Strategy:* backtracking + MRV/degree + forward checking/AC-3, with min-conflicts local search to repair a timetable. Treat hard constraints (must hold) separately from soft ones (preferences). This scales far better than plain backtracking.

5. Strategic game AI (Minimax and Alpha-Beta)

i. Minimax model

- The game is a tree with MAX nodes (the AI) and MIN nodes (the human). Leaf values come from a utility/evaluation function. MAX takes the child with the highest value and MIN the lowest, so the AI assumes the opponent plays optimally.

ii. Computational challenges

- Time is $O(b^d)$ with a huge branching factor (many units and actions); depth is limited by the time per move, so full search is impossible. Hidden information and real-time play add difficulty.

iii. Alpha-Beta pruning

- α is the best value MAX can guarantee so far and β the best value MIN can guarantee. If $\alpha \geq \beta$ at a node, the remaining children cannot change the final decision and are pruned.
- The result is identical to Minimax. With good move ordering the best case drops from $O(b^d)$ to $O(b^{d/2})$, roughly doubling the searchable depth.

iv. Evaluation function

- Eval(s) = w1·(unit/material strength difference) + w2·(resource income) + w3·(map and position control) + w4·(base health) + w5·(tech/upgrade progress) + w6·(threat/safety).
- Each factor reflects a part of winning: military strength, economy, territory, survival and long-term growth; the weights can be tuned from experience.

v. Strategy

- Depth-limited alpha-beta with iterative deepening, so the best move found so far is always available when time runs out. Add move ordering (best moves first), a transposition table (cache of seen states) and a good evaluation function at the cut-off; optionally selective deepening for tactical moves. This balances optimality and the time limit.

RUBRICS FOR CALCULATION

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand